

Primary Analysis

Jonathan Hwang

2026-02-12

Initialization

```
renv::init()

library(tidyverse)
library(rootSolve)
library(data.table)
library(msos)
library(entropy)
library(simboot)
library(MCPAN)
library(boot)
library(gridExtra)
library(cowplot)
library(ggpubr)
library(progress)
library(renv)
library(gridExtra)
library(Hmisc)
library(lubridate)
library(tmap)
library(ggplotify)
library(sf)
sf_use_s2(FALSE)

#Clear environment
rm(list = ls())

knitr::opts_chunk$set(echo = TRUE)
```

First, we import all of the csv files.

```
#Mortality data for all parishes from January 1, 1629 to December 31, 1631.
all <- read.csv("ALL_utf.csv")
city_peaks <- read.csv("city_cases_baseline_removed.csv")
filled <- read.csv("filledna_full_ts.csv")

#Geographical data for locations of Venetian parishes
geo_df <- read.csv("geo_df.csv")

#Venetian census data from 1696
population <- read.csv("ParishPopulations.csv")
```

```
#Shows which parishes belong to which sestieri
sestiere_pre <- read.csv("sestiereDataPre.csv")
sestiere_post <- read.csv("SestiereDataPost.csv")
```

Then, we perform preliminary cleaning.

```
#Fix the Dates on all
all$Date <- sub("2029", "1629", all$Date)
all$Date <- sub("2030", "1630", all$Date)
all$Date <- sub("2031", "1631", all$Date)
all$Date <- as.Date(all$Date, format="%m/%d/%Y")

#Fix artifacts for column names
colnames(all) <- sub("S.", "S", colnames(all))

for(column in 1:ncol(all)){
  if(column == 1){
    next
  }
  if(!(is.integer(all[,column]))){
    all[,column] = as.integer(all[,column])
  }
}

#Fix the Dates on filled
colnames(filled)[1] <- "Date"
filled$Date <- as.Date(filled$Date, format="%Y-%m-%d")

colnames(filled) <- sub("S.", "S", colnames(filled))

for(column in 1:ncol(filled)){
  if(column == 1){
    next
  }
  if(!(is.integer(filled[,column]))){
    filled[,column] = as.integer(filled[,column])
  }
}

#Fix the Dates on city_peaks
colnames(city_peaks)[1] <- "Date"
city_peaks$Date <- sub(" 00:00:00", "", city_peaks$Date)
city_peaks$Date <- as.Date(city_peaks$Date, format="%Y-%m-%d")

#Fix geo_df
colnames(geo_df)[1] <- "Parish"
geo_df$Parish <- gsub("[:space:]", ".", geo_df$Parish)
geo_df$Parish <- sub("S.", "S", geo_df$Parish)

#Get rid of Angelo.Rafael because it has no lat/long data
geo_df <- geo_df[!grepl("Angelo.Rafael", geo_df$Parish),]

#Fix population
population$Parish <- gsub("[:space:]", ".", population$Parish)
population$Parish <- sub("S.", "S", population$Parish)
```

```

#What are the missing cities in geo_df?
setdiff(geo_df$Parish, colnames(filled))

## [1] "S.Giacomo.dell'Orio"

#What are the missing cities in population?
filled <- filled[, -match(c("S.Marcilian"), colnames(filled))] #Marcilian exists in the
↪ population_pre dataset.

all_nas <- c()
for(col in 1:ncol(filled)){
  if(sum(is.na(filled[,col])) == length(filled[,col])){
    all_nas <- c(all_nas, col)
  }
}
if(length(all_nas) > 0){
  filled <- filled[, -all_nas]
}

missing_parishes <- c()
for(p in 1:length(colnames(filled))){
  if(!(colnames(filled)[p] %in% population$Parish) & colnames(filled)[p] != "Date"){
    missing_parishes <- c(missing_parishes,p)
  }
}
if(length(missing_parishes) > 0){
  filled <- filled[, -missing_parishes]
}

```

The code below removes San Marco from our analysis. In our main analysis, we exclude it, but in the supplementary information, we include San Marco as a form of sensitivity analysis.

```

#Get rid of S. Marco, the outlier
filled <- filled[, -match("S.Marco", colnames(filled))]

```

Import csv files containing population data from Venetian censuses in 1509, 1551, 1581, 1586, and 1696 for each parish.

```

population_pre <- read.csv("ParishPopulationsPre.csv")
population_post <- read.csv("ParishPopulationsPost.csv")

```

Clean the population datasets.

```

colnames(population_pre) <- c("Parish", "Population_1509", "Population_1551",
↪ "Population_1581", "Population_1586")

#Remove artifacts
population_pre$Parish <- gsub("[[:space:]]", ".", population_pre$Parish)
population_pre$Parish <- sub("S.", "S", population_pre$Parish)

population_post$Parish <- gsub("[[:space:]]", ".", population_post$Parish)
population_post$Parish <- sub("S.", "S", population_post$Parish)

#Identify and remove the parishes that are not contained in both the 1586 census and the
↪ 1696 census

```

```

remove_vec <- c()
for(i in 1:length(population_pre$Parish)){
  if(!(population_pre$Parish[i] %in% population_post$Parish)){
    remove_vec <- c(remove_vec,i)
  }
}

if(length(remove_vec) > 0){
  population_pre <- population_pre[-remove_vec,]
}

remove_vec <- c()
for(i in 1:length(population_post$Parish)){
  if(!(population_post$Parish[i] %in% population_pre$Parish)){
    remove_vec <- c(remove_vec, i)
  }
}

if(length(remove_vec) > 0){
  population_post <- population_post[-remove_vec,]
}

#Establish a consistent ordering of the parishes (alphabetical)
population_post <- population_post %>% arrange(Parish)
population_pre <- population_pre %>% arrange(Parish)

population_pre <- population_pre[,-2]

for(j in 1:ncol(population_pre)){
  if(j == 1){
    next
  }
  for(i in 1:nrow(population_pre)){
    population_pre[i,j] <- format(as.numeric(population_pre[i,j]),nsmall=3)
    if(nchar(population_pre[i,j]) < 7){
      population_pre[i,j] <- as.numeric(population_pre[i,j]) * 1000
    } else {
      population_pre[i,j] <- as.numeric(population_pre[i,j])
    }
  }
  population_pre[,j] <- as.numeric(population_pre[,j])
}
population_post$Population <- as.numeric(population_post$Population) * 1000

#Merge population_pre and population_post into one dataframe, population_df
population_df <- as.data.frame(cbind(population_pre, population_post[, -1]))
colnames(population_df) <- c("Parish", "Population_1551", "Population_1581",
  ↪ "Population_1586", "Population_1696")

#Plot the population data over time
year_vec <- c(c(1551, 1581, 1586, 1696))

t_pop_df <- transpose(population_df)
rownames(t_pop_df) <- colnames(population_df)

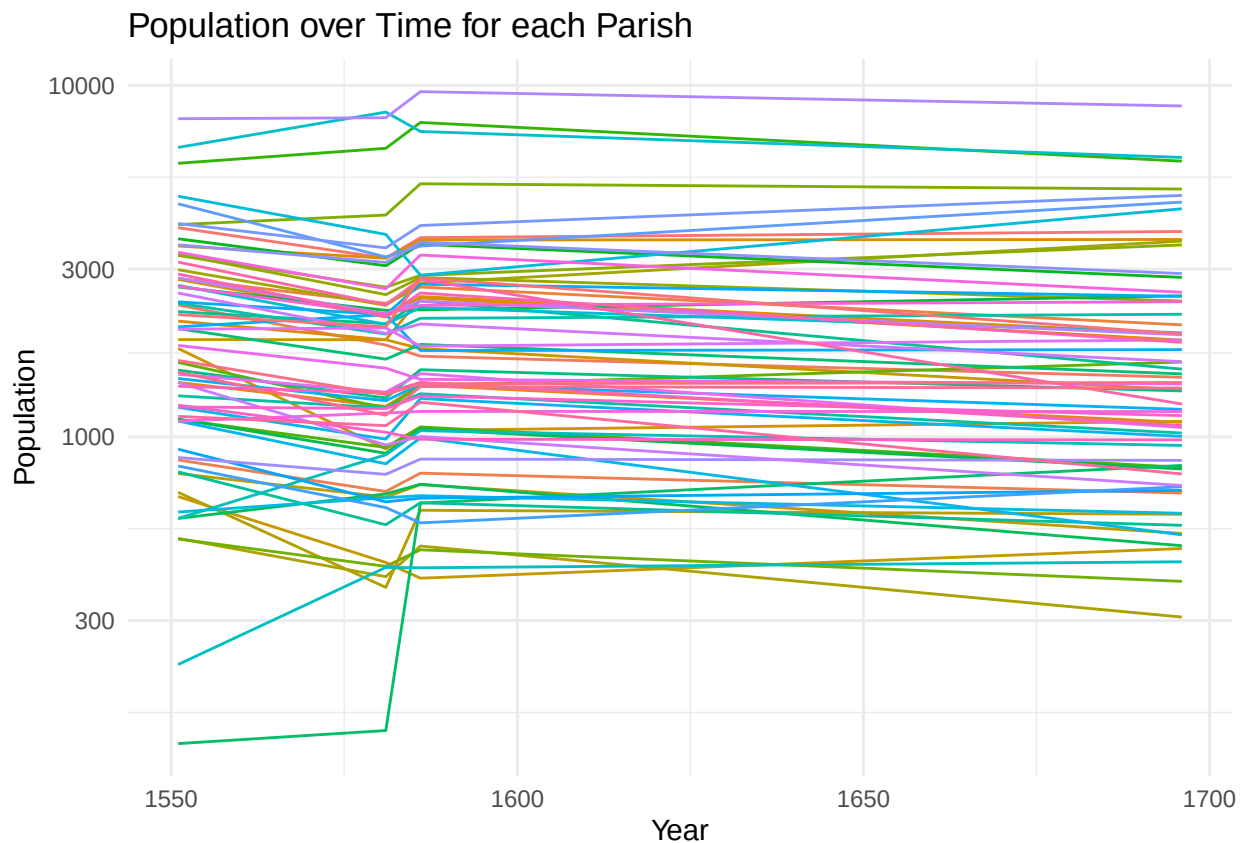
```

```

t_pop_df <- t_pop_df[-1,]
colnames(t_pop_df) <- population_df$Parish
t_pop_df <- cbind(year_vec, t_pop_df)
t_pop_df_plot <- t_pop_df %>%
  gather(key="Parish_var", value = "value", -year_vec)
t_pop_df_plot$value <- as.numeric(t_pop_df_plot$value)

t_pop_df_plot %>%
  ggplot(aes(x = year_vec, y = value))+
  geom_line(aes(color = Parish_var), show.legend = FALSE, linewidth = 0.5)+
  labs(x = "Year", y = "Population", title = "Population over Time for each Parish")+
  scale_y_log10()+
  theme_minimal()

```



```

ggsave("PopulationsOverTimeNoLegendYLog10.png", width = 1920, height = 1080, units =
  ↪ "px")

```

Plot the 1696 population data vs. the 1586 population data.

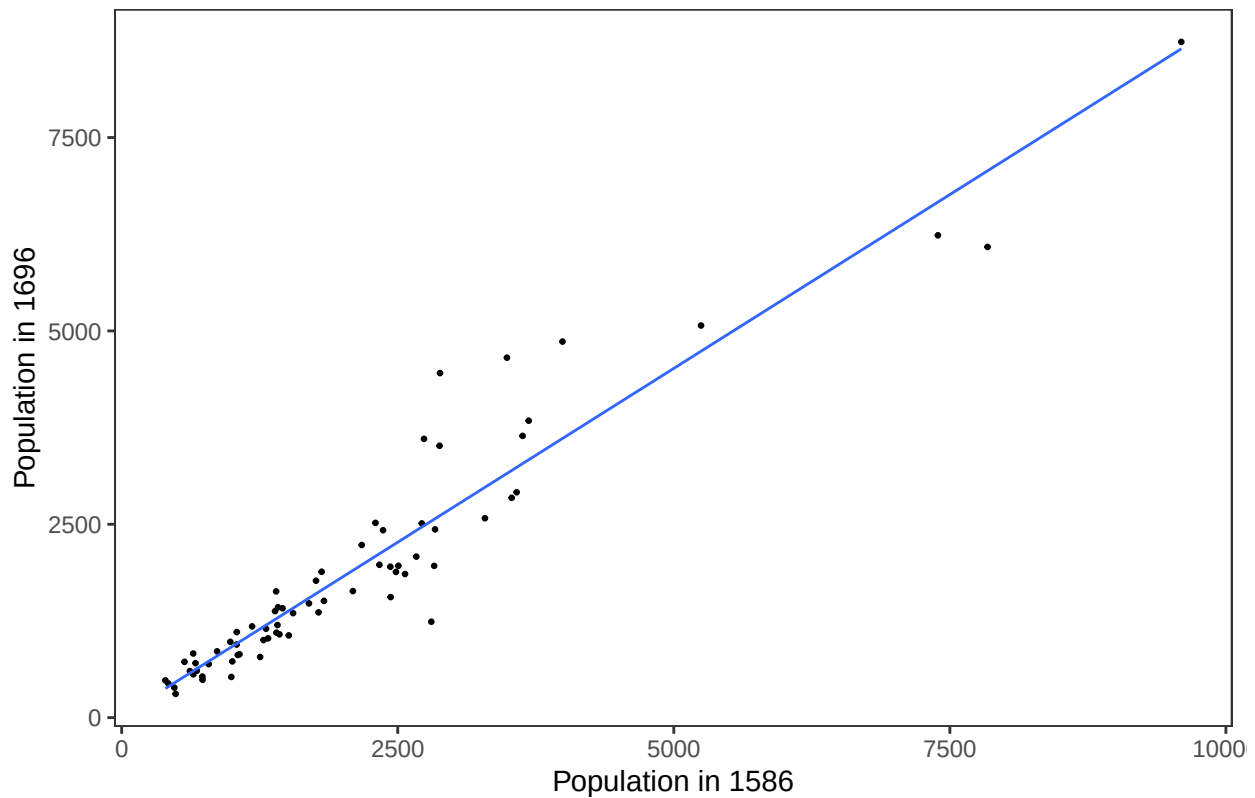
```

population_df %>%
  ggplot(aes(x = Population_1586, y = Population_1696))+
  geom_point(size = 0.5)+
  geom_smooth(method = lm, se = FALSE, linewidth = 0.5)+
  theme_bw()+
  theme(text = element_text(family = ""), panel.grid.major = element_blank(),
  ↪ panel.grid.minor = element_blank())+
  labs(x = "Population in 1586", y = "Population in 1696", title = "Population in 1696
  ↪ vs. Population in 1586")

```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Population in 1696 vs. Population in 1586



```
#Save as Supplementary Figure 1
```

```
ggsave("SuppFig1.jpg", width = 1920, height = 1080, units = "px")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

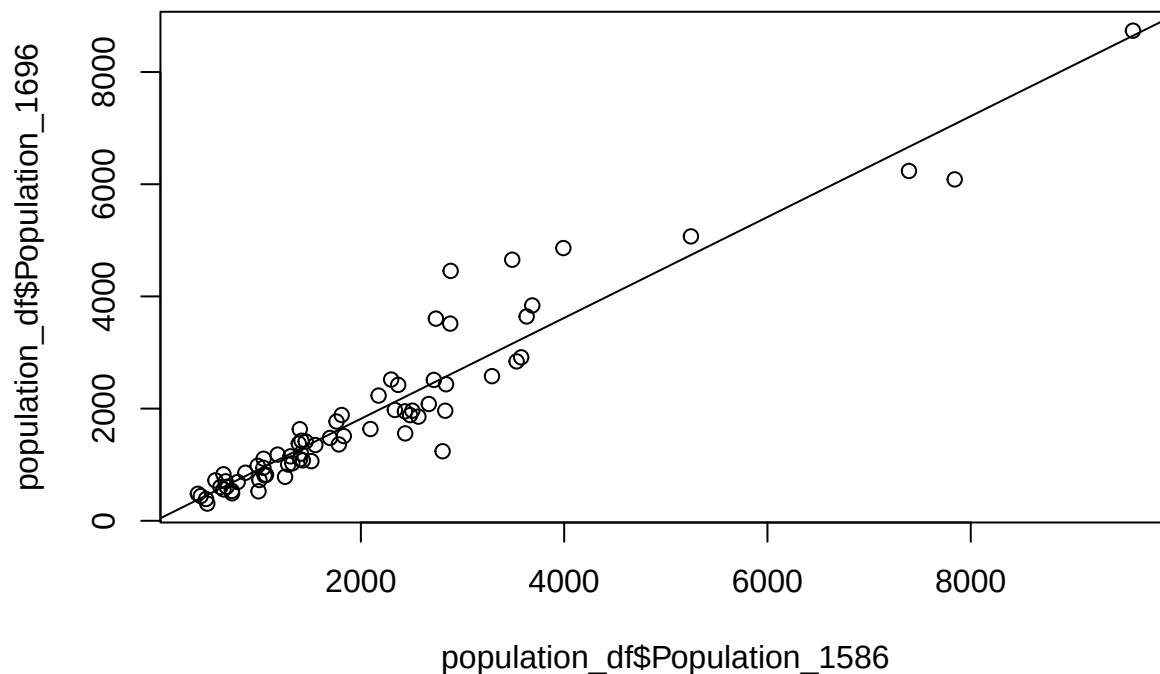
Run a linear regression on the Venetian census data from 1696 vs. 1586 to determine whether or not linear interpolation is a valid approach.

```
cor(population_df$Population_1586, population_df$Population_1696) #0.95
```

```
## [1] 0.9524433
```

```
plot(x = population_df$Population_1586, y = population_df$Population_1696)
```

```
abline(mod <- lm(population_df$Population_1696 ~ population_df$Population_1586)) #y =  
↪ 1696, x = 1586 (closest census date) so slope smaller than 1 means that population  
↪ decreased over time
```



```
intercept <- as.numeric(coef(mod)[1])
slope <- as.numeric(coef(mod)[2]) #slope, close to 1 so we feel comfortable interpolating
```

Due to the high r value (0.95), we feel comfortable interpolating between the 1696 and 1586 population data to estimate each parish's population on January 1, 1629.

```
interpolated_1629 <- (intercept - (1-slope) * population_pre$Population_1586) * 43/110 +
  ↪ population_pre$Population_1586
```

Create `population_specified`, a dataframe containing only the populations of the parishes of interest.

```
population <- data.frame(population_post$Parish, interpolated_1629 / 1000)
colnames(population) <- c("Parish", "Population")
population_specified <- population[population$Parish %in% colnames(filled[, -1]),]
population_specified <- population_specified[match(colnames(filled[, -1]),
  ↪ population_specified$Parish),]
```

Now, we combine `sestiere_pre` and `sestiere_post`, the two dataframes that list the parishes within each sestiere (a sestiere is a larger geographical area that contains multiple parishes).

```
#geo_df doesn't have S.Giacomo.dell'Orio and Angelo.Rafael is blank.

#Fix the dates on sestiere_dfs
sestiere_pre$Parish <- gsub("[[:space:]]", ".", sestiere_pre$Parish)
sestiere_pre$Parish <- sub("S.", "S", sestiere_pre$Parish)
sestiere_post$Parish <- gsub("[[:space:]]", ".", sestiere_post$Parish)
sestiere_post$Parish <- sub("S.", "S", sestiere_post$Parish)

#arrange grouping by sestiere
sestiere_pre <- sestiere_pre %>% arrange(Sestiere, Parish)
sestiere_post <- sestiere_post %>% arrange(Sestiere, Parish)

#Match parishes in sestiere_df with those in filled
sestiere_df <- sestiere_pre
```

```

remove_vec <- c()
for(i in 1:length(sestiere_df$Parish)){
  if(!(sestiere_df$Parish[i] %in% colnames(filled[,,-1]))){
    remove_vec <- c(remove_vec,i)
  }
}

if(length(remove_vec) > 0){
  sestiere_df <- sestiere_df[-remove_vec,]
}

#Define dataframes for each individual year
filled_1629 <- filled[year(filled$Date) == 1629,]
filled_1630 <- filled[year(filled$Date) == 1630,]
filled_1631 <- filled[year(filled$Date) == 1631,]

```

Finally, we figure out which parishes we will keep. Specifically, we exclude the parishes which had no data whatsoever in the entire primary outbreak window (9/1/30-12/31/30).

```

outliers <- c("S.Polo", "S.Stae", "S.Marco") #No cases between 9/1/30 and 12/31/30,
  ↪ outbreak time
filled_modified <- filled[, !(colnames(filled) %in% outliers)]
sestiere_df_modified <- sestiere_df[!(sestiere_df$Parish %in% outliers),]
population_specified_modified <- population_specified[!(population_specified$Parish %in%
  ↪ outliers),]

```

Bootstrapping

In this section of code, we run bootstrapping ($n = 999$) to identify 95% and 5% confidence intervals for all of the statistics we calculate.

```

library(progress)
library(dplyr)
library(ggplot2)
library(gridExtra)
library(cowplot)
library(ggpubr)
library(tidyverse)
library(lubridate)
library(msos)

```

First, we define the functions we will (manually) bootstrap:

For our histogram approximation of the PDF, we use Sturges's Rule to choose K , the number of bins, for n observations ($K = \log_2(n) + 1$).

```

#Define a function that provides a histogram approximation of a PDF with K bins. Here, we
  ↪ use Sturges's Rule for K.
PDF_approx <- function(x, K = ceiling(log2(length(x))) + 1){
  p <- table(cut(x, breaks = K)) / length(x)
  sigma2 <- sum((1:K)^2 * p) - sum((1:K) * p)^2
  p <- p[(p>0)]
  return(p)
}

```

```
PDF_approx_2 <- function(x,K = ceiling(log2(length(x))) + 1){
  p <- table(cut(x,breaks = K)) / length(x)
  return(p)
}
```

To calculate Kullback-Leibler divergence between two discrete probability distributions $P(x)$ and $Q(x)$, we use the definition $\sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$. While this formulation implies that KL divergence is undefined when $P(x) = Q(x) = 0$, we treat the contribution of the KL divergence at that point as a zero (https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence).

```
#Next, we define Kullback-Leibler divergence between two discrete probability
↪ distributions (x and y), formatted as vectors. We add a small value epsilon to avoid
↪ dividing by zero.
KL_new <- function(x, y, epsilon = 1e-05){
  distance = 0
  pq_ratio = 0
  if(length(x) !=length(y)){
    print("not same length")
    return(NA)
  } else {
    for(i in 1:length(x)){
      if(is.nan(x[i]) | is.nan(y[i])){
        distance = distance + 0
      } else {
        if(x[i] == 0 && y[i] == 0){
          distance = distance + 0
        } else{
          if(y[i] == 0){
            pq_ratio=x[i] / epsilon
          } else{
            pq_ratio = x[i] / y[i]
          }
          if(pq_ratio == 0){
            distance = distance + 0
          } else{
            distance = distance + x[i] * log(pq_ratio)
          }
        }
      }
    }
  }
  return(distance)
}
```

Now we define our master analysis function which returns the total population, Shannon entropy (H) of the PDF $P(x)$ ($H = -\sum_{x \in \mathcal{X}} P(x) \log P(x)$), Shannon Entropy of the quantile function, negentropy of the PDF (KL divergence with a Gaussian), KL divergence of the PDF from a uniform, and KL divergence of the quantile from a discrete uniform at a given timestep. Later, we will run this function on the 999 resampled datasets generated during the bootstrapping process.

```
entropyprocedure_sestiere_3 <- function(original_data, subset, indices, timestep,
  ↪ cumulative = TRUE){
  boot_data <- original_data[, indices]
  pop_spec <- population_specified_modified[subset - 1,][indices,]
  if(cumulative){
```

```

    non_pop_corrected <- as.numeric(colSums(boot_data[1:timestep,]))
  } else {
    non_pop_corrected <- as.numeric(boot_data[timestep,])
  }
  cv <- non_pop_corrected / (pop_spec$Population * 1000)
  cv <- sort(cv)
  cv_norm <- cv / sum(cv)
  kern <- as.numeric(PDF_approx(cv))
  #kern <- as.numeric(PDF_approx_2(cv))
  kern_norm <- kern / sum(kern)
  shannon_PDF <- -1 * sum(kern[kern != 0] * log(kern[kern != 0]))
  shannon_quantile <- -1 * sum(cv_norm[cv_norm != 0] * log(cv_norm[cv_norm != 0]))
  negent_PDF <- negent(cv)
  #negent_PDF <- negent_new(cv)
  uniform_KL_PDF <- as.numeric(KL_new(kern, rep(1 / length(kern), length(kern))))
  uniform_KL_quantile <- KL_new(cv_norm, rep(1 / length(cv_norm), length(cv_norm)))
  return(list(sum = sum(non_pop_corrected) / (sum(pop_spec$Population) * 1000),
    ↪ shannonPDF = shannon_PDF, shannonQuantile = shannon_quantile, negentPDF =
    ↪ negent_PDF, uniformKL_PDF = uniform_KL_PDF, uniformKL_Quantile =
    ↪ uniform_KL_quantile))
}

manual_boot_modified_3 <- function(data, subset, N, max_ts, cumulative) {
  sum_mat <- matrix(NA, nrow = N, ncol = max_ts)
  shannonPDF_mat <- matrix(NA, nrow = N, ncol = max_ts)
  shannonQuantile_mat <- matrix(NA, nrow = N, ncol = max_ts)
  negentPDF_mat <- matrix(NA, nrow = N, ncol = max_ts)
  uniformKL_PDF_mat <- matrix(NA, nrow = N, ncol = max_ts)
  uniformKL_Quantile_mat <- matrix(NA, nrow = N, ncol = max_ts)
  for (i in 1:N) {
    sample_index <- sample(ncol(data), replace = TRUE)
    for (t in 1:max_ts) {
      current_list <- entropyprocedure_sestiere_3(data, subset, sample_index,
    ↪ t, cumulative)
      sum_mat[i,t] <- current_list$sum
      shannonPDF_mat[i,t] <- current_list$shannonPDF
      shannonQuantile_mat[i,t] <- current_list$shannonQuantile
      negentPDF_mat[i,t] <- current_list$negentPDF
      uniformKL_PDF_mat[i,t] <- current_list$uniformKL_PDF
      uniformKL_Quantile_mat[i,t] <- current_list$uniformKL_Quantile
    }
  }

  ↪ return(list(sum=sum_mat, shannonPDF=shannonPDF_mat, shannonQuantile=shannonQuantile_mat, negentPDF
  ↪ = negentPDF_mat, uniformKL_PDF = uniformKL_PDF_mat, uniformKL_Quantile =
  ↪ uniformKL_Quantile_mat))
}

#Instead of finding the 95th and 5th percentile full runs, we find the 9th and 5th
↪ percentile values at each time point.

find_closest_run <- function(all_runs, target_percentile) {
  all_runs_sum <- rowSums(all_runs)
  sorted_all_runs_sum <- sort(all_runs_sum)

```

```

    index <- match(sorted_all_runs_sum[target_percentile], all_runs_sum)
    return(as.numeric(all_runs[index,]))
}

shannonBySestiere_Bootstrapped_4 <- function(cumulative = TRUE) {
  num_dates <- length(filled_modified$Date)
  sestiere_list <- unique(sestiere_df_modified$Sestiere)
  num_sestieri <- length(sestiere_list)
  st <- Sys.time()
  all_runs_list <- vector(mode = "list", length = length(sestiere_list))
  for (s in sestiere_list) {
    cat("Processing:", s, "\n")
    s_index <- match(s, sestiere_list)
    sestiere_parishes <- match(sestiere_df_modified$Parish[sestiere_df_modified$Sestiere
↪ == s], colnames(filled_modified))

    data_subset <- filled_modified[, sestiere_parishes]

    boot_mat <- manual_boot_modified_3(data = data_subset, subset = sestiere_parishes, N
↪ = 999, max_ts = num_dates, cumulative = cumulative)

    b1_mat <- matrix(NA, ncol = length(boot_mat), nrow = num_dates)
    b2_mat <- matrix(NA, ncol = length(boot_mat), nrow = num_dates)
    closest_runs_025_mat <- matrix(NA, ncol = length(boot_mat), nrow = num_dates)
    closest_runs_975_mat <- matrix(NA, ncol = length(boot_mat), nrow = num_dates)
    #b1[, s_index] <- apply(boot_mat, 2, quantile, probs = 0.025, na.rm = TRUE)
    #b2[, s_index] <- apply(boot_mat, 2, quantile, probs = 0.975, na.rm = TRUE)
    for(j in 1:length(boot_mat)){
      closest_runs_025_mat[,j] <- find_closest_run(boot_mat[[j]], 25)
      closest_runs_975_mat[,j] <- find_closest_run(boot_mat[[j]], 975)
      for(i in 1:ncol(boot_mat[[j]])){
        sorted <- sort(as.numeric(boot_mat[[j]][,i]))
        b1_mat[i, j] <- sorted[25]
        b2_mat[i, j] <- sorted[975]
      }
    }
    all_runs_list[[match(s,sestiere_list)]] <- list(b1 = b1_mat, b2 = b2_mat,
↪ closest_runs_025 = closest_runs_025_mat, closest_runs_975 = closest_runs_975_mat)
  }
  ed <- Sys.time()
  print(ed - st)
  return(all_runs_list)
}

shannonBySestiere_4 <- function(cumulative_boolean = TRUE){
  sum_vec <- rep(0, length(filled_modified$Date))
  shannonPDF_vec <- rep(0, length(filled_modified$Date))
  shannonQuantile_vec <- rep(0, length(filled_modified$Date))
  negentPDF_vec <- rep(0, length(filled_modified$Date))
  uniformKL_PDF_vec <- rep(0, length(filled_modified$Date))
  uniformKL_Quantile_vec <- rep(0, length(filled_modified$Date))
  all_runs_list <- vector(mode = "list", length =
↪ length(unique(sestiere_df_modified$Sestiere)))
  for(s in unique(sestiere_df_modified$Sestiere)){

```

```

sestiere_parishes <- match(sestiere_df_modified$Parish[sestiere_df_modified$Sestiere
→ == s], colnames(filled_modified))
for(i in 1:length(filled_modified$Date)){
  current_list <- entropyprocedure_sestiere_3(filled_modified[,sestiere_parishes],
→ subset = sestiere_parishes, seq(1, length(sestiere_parishes), length.out =
→ length(sestiere_parishes)), timestep = i, cumulative = cumulative_boolean)
  sum_vec[i] <- current_list$sum
  shannonPDF_vec[i] <- current_list$shannonPDF
  shannonQuantile_vec[i] <- current_list$shannonQuantile
  negentPDF_vec[i] <- current_list$negentPDF
  uniformKL_PDF_vec[i] <- current_list$uniformKL_PDF
  uniformKL_Quantile_vec[i] <- current_list$uniformKL_Quantile
}
all_runs_list[[match(s,unique(sestiere_df_modified$Sestiere))]] <- list(sum =
→ sum_vec, shannonPDF = shannonPDF_vec, shannonQuantile = shannonQuantile_vec,
→ negentPDF = negentPDF_vec, uniformKL_PDF = uniformKL_PDF_vec, uniformKL_Quantile =
→ uniformKL_Quantile_vec)
}
return(all_runs_list)
}

whole_dataset_analysis <- function(cumulative = TRUE){
  sum_vec <- rep(0, length(filled_modified$Date))
  shannonPDF_vec <- rep(0, length(filled_modified$Date))
  shannonQuantile_vec <- rep(0, length(filled_modified$Date))
  negentPDF_vec <- rep(0, length(filled_modified$Date))
  uniformKL_PDF_vec <- rep(0, length(filled_modified$Date))
  uniformKL_Quantile_vec <- rep(0, length(filled_modified$Date))
  st <- Sys.time()
  pb <- progress_bar$new(total = length(filled_modified$Date))
  for(i in 1:length(filled_modified$Date)){
    current_list <- entropyprocedure_sestiere_3(filled_modified[, -1], subset =
→ 2:ncol(filled_modified), indices = 1:ncol(filled_modified[, -1]), timestep =
→ i, cumulative)
    sum_vec[i] <- current_list$sum
    shannonPDF_vec[i] <- current_list$shannonPDF
    shannonQuantile_vec[i] <- current_list$shannonQuantile
    negentPDF_vec[i] <- current_list$negentPDF
    uniformKL_PDF_vec[i] <- current_list$uniformKL_PDF
    uniformKL_Quantile_vec[i] <- current_list$uniformKL_Quantile
    pb$tick()
  }
  ed <- Sys.time()
  print(ed - st)
  return(list(sum = sum_vec, shannonPDF = shannonPDF_vec, shannonQuantile =
→ shannonQuantile_vec, negentPDF = negentPDF_vec, uniformKL_PDF = uniformKL_PDF_vec,
→ uniformKL_Quantile = uniformKL_Quantile_vec))
}

whole_dataset_bootstrap <- function(cumulative = TRUE){
  num_dates <- length(filled_modified$Date)
  b1 <- rep(0, num_dates) # 2.5%
  b2 <- rep(0, num_dates) # 97.5%
  boot_mat <- manual_boot_modified_3(data = filled_modified[, -1], subset =
→ 2:ncol(filled_modified), N = 999, max_ts = num_dates, cumulative = cumulative)

```

```

b1_list <- vector(mode = "list", length = length(boot_mat))
b2_list <- vector(mode = "list", length = length(boot_mat))
closest_runs_025_list <- vector(mode = "list", length(boot_mat))
closest_runs_975_list <- vector(mode = "list", length(boot_mat))
#b1[, s_index] <- apply(boot_mat, 2, quantile, probs = 0.025, na.rm = TRUE)
#b2[, s_index] <- apply(boot_mat, 2, quantile, probs = 0.975, na.rm = TRUE)
for(j in 1:length(boot_mat)){
  closest_025 <- find_closest_run(boot_mat[[j]], 25)
  closest_975 <- find_closest_run(boot_mat[[j]], 975)
  for(i in 1:ncol(boot_mat[[j]])){
    sorted <- sort(as.numeric(boot_mat[[j]][,i]))
    b1[i] <- sorted[25]
    b2[i] <- sorted[975]
  }
  b1_list[[j]] <- b1
  b2_list[[j]] <- b2
  closest_runs_025_list[[j]] <- closest_025
  closest_runs_975_list[[j]] <- closest_975
}
return(list(b1 = b1_list, b2 = b2_list, closest_runs_025 = closest_runs_025_list,
  ↪ closest_runs_975 = closest_runs_975_list))
}

```

Finally, we run our bootstrap!

```

stats_by_sestiere_boot_cumulative_50_2 <- shannonBySestiere_Bootstrapped_4(cumulative =
  ↪ TRUE) #w/ fixed negent
stats_by_sestiere_boot_instantaneous_50_2 <- shannonBySestiere_Bootstrapped_4(cumulative
  ↪ = FALSE)
stats_by_sestiere_cumulative_50_2 <- shannonBySestiere_4(cumulative = TRUE)
stats_by_sestiere_instantaneous_50_2 <- shannonBySestiere_4(cumulative = FALSE)
stats_of_whole_cumulative_50_2 <- whole_dataset_analysis(cumulative = TRUE) #default
stats_of_whole_instantaneous_50_2 <- whole_dataset_analysis(cumulative = FALSE)
stats_of_whole_boot_cumulative_50_2 <- whole_dataset_bootstrap(cumulative = TRUE)
stats_of_whole_boot_instantaneous_50_2 <- whole_dataset_bootstrap(cumulative = FALSE)

#Since we don't use a set seed, save our bootstrap values to a separate document
#
↪ saveRDS(stats_by_sestiere_boot_cumulative_50_2, "stats_by_sestiere_boot_cumulative_50_2.rds")
#
↪ saveRDS(stats_by_sestiere_boot_instantaneous_50_2, "stats_by_sestiere_boot_instantaneous_50_2.rds")
# saveRDS(stats_of_whole_boot_cumulative_50_2, "stats_of_whole_boot_cumulative_50_2.rds")
#
↪ saveRDS(stats_of_whole_boot_instantaneous_50_2, "stats_of_whole_boot_instantaneous_50_2.rds")

```

Now we plot our results.

First, we plot the Shannon entropy within each sestiere over time along with the 5% and 95% confidence intervals generated from bootstrapping.

```

stats_of_whole_cumulative_50_2$shannonQuantile[1] #1.988281
stats_of_whole_boot_cumulative_50_2$b2[[3]][1] #2.530452
stats_of_whole_boot_cumulative_50_2$b1[[3]][1] #1.236387
stats_of_whole_cumulative_50_2$shannonQuantile[1095] #3.855849
stats_of_whole_boot_cumulative_50_2$b1[[3]][1095] #3.841083
stats_of_whole_boot_cumulative_50_2$b2[[3]][1095] #3.868945

```

```

stats_of_whole_cumulative_50_2$uniformKL_Quantile[1] #1.90354
stats_of_whole_boot_cumulative_50_2$b1[[6]][1] #1.361368
stats_of_whole_boot_cumulative_50_2$b2[[6]][1] #2.655433
stats_of_whole_cumulative_50_2$uniformKL_Quantile[1095] #0.03597084
stats_of_whole_boot_cumulative_50_2$b2[[6]][1095] #0.050_273722
stats_of_whole_boot_cumulative_50_2$b1[[6]][1095] #0.02287577

stats_of_whole_cumulative_50_2$shannonPDF[1] #0.6430871
stats_of_whole_boot_cumulative_50_2$b1[[2]][1] #0.3393223
stats_of_whole_boot_cumulative_50_2$b2[[2]][1] #0.9709571
stats_of_whole_cumulative_50_2$shannonPDF[1095] #1.738919
stats_of_whole_boot_cumulative_50_2$b1[[2]][1095] #1.538075
stats_of_whole_boot_cumulative_50_2$b2[[2]][1095] #1.839712

stats_of_whole_cumulative_50_2$negentPDF[1] #0.8352903
stats_of_whole_boot_cumulative_50_2$b2[[4]][1] #1.441464
stats_of_whole_boot_cumulative_50_2$b1[[4]][1] #0.7284727
stats_of_whole_cumulative_50_2$negentPDF[1095] #0.1072035
stats_of_whole_boot_cumulative_50_2$b1[[4]][1095] #0.07247547
stats_of_whole_boot_cumulative_50_2$b2[[4]][1095] #0.3258044

stats_of_whole_cumulative_50_2$uniformKL_PDF[1] #0.966350_28
stats_of_whole_boot_cumulative_50_2$b1[[5]][1] #0.5657428
stats_of_whole_boot_cumulative_50_2$b2[[5]][1] #1.170918
stats_of_whole_cumulative_50_2$uniformKL_PDF[1095] #0.2069911
stats_of_whole_boot_cumulative_50_2$b2[[5]][1095] #0.3832459
stats_of_whole_boot_cumulative_50_2$b1[[5]][1095] #0.08352609

stats_of_whole_boot_cumulative_50_2$closest_runs_025[[5]][1095] #0.1674439
stats_of_whole_boot_cumulative_50_2$closest_runs_975[[5]][1095] #0.2010526

min(stats_of_whole_cumulative_50_2$uniformKL_PDF) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_cumulative_50_2$uniformKL_PDF),stats_of_whole_cumulative_50_2$uniformKL_PDF)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_cumulative_50_2$uniformKL_PDF[min_index]
stats_of_whole_boot_cumulative_50_2$b1[[5]][min_index]
stats_of_whole_boot_cumulative_50_2$b2[[5]][min_index]

min(stats_of_whole_cumulative_50_2$negentPDF) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_cumulative_50_2$negentPDF),stats_of_whole_cumulative_50_2$negentPDF)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_cumulative_50_2$negentPDF[min_index]
stats_of_whole_boot_cumulative_50_2$b1[[4]][min_index]
stats_of_whole_boot_cumulative_50_2$b2[[4]][min_index]

min(stats_of_whole_cumulative_50_2$uniformKL_Quantile) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_cumulative_50_2$uniformKL_Quantile),stats_of_whole_cumulative_50_2$uniformKL_Quantile)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_cumulative_50_2$uniformKL_Quantile[min_index]
stats_of_whole_boot_cumulative_50_2$b1[[6]][min_index]

```

```

stats_of_whole_boot_cumulative_50_2$b2[[6]][min_index]

max(stats_of_whole_cumulative_50_2$shannonQuantile) #0.1382115
max_index <-
  ↳ match(max(stats_of_whole_cumulative_50_2$shannonQuantile),stats_of_whole_cumulative_50_2$shannonQuantile)
filled$Date[max_index] #"1631-06-17"
stats_of_whole_cumulative_50_2$shannonQuantile[max_index]
stats_of_whole_boot_cumulative_50_2$b1[[3]][max_index]
stats_of_whole_boot_cumulative_50_2$b2[[3]][max_index]

max(stats_of_whole_cumulative_50_2$shannonPDF) #0.1382115
max_index <-
  ↳ match(max(stats_of_whole_cumulative_50_2$shannonPDF),stats_of_whole_cumulative_50_2$shannonPDF)
filled$Date[max_index] #"1631-06-17"
stats_of_whole_cumulative_50_2$shannonPDF[max_index]
stats_of_whole_boot_cumulative_50_2$b1[[2]][max_index]
stats_of_whole_boot_cumulative_50_2$b2[[2]][max_index]

stats_of_whole_instantaneous_50_2$shannonQuantile[1] #1.988281
stats_of_whole_boot_instantaneous_50_2$b2[[3]][1] #2.530452
stats_of_whole_boot_instantaneous_50_2$b1[[3]][1] #1.236387
stats_of_whole_instantaneous_50_2$shannonQuantile[1095] #3.855849
stats_of_whole_boot_instantaneous_50_2$b1[[3]][1095] #3.841083
stats_of_whole_boot_instantaneous_50_2$b2[[3]][1095] #3.868945

stats_of_whole_instantaneous_50_2$uniformKL_Quantile[1] #1.90354
stats_of_whole_boot_instantaneous_50_2$b1[[6]][1] #1.361368
stats_of_whole_boot_instantaneous_50_2$b2[[6]][1] #2.655433
stats_of_whole_instantaneous_50_2$uniformKL_Quantile[1095] #0.03597084
stats_of_whole_boot_instantaneous_50_2$b2[[6]][1095] #0.050_273722
stats_of_whole_boot_instantaneous_50_2$b1[[6]][1095] #0.02287577

stats_of_whole_instantaneous_50_2$shannonPDF[1] #0.6430871
stats_of_whole_boot_instantaneous_50_2$b1[[2]][1] #0.3393223
stats_of_whole_boot_instantaneous_50_2$b2[[2]][1] #0.9709571
stats_of_whole_instantaneous_50_2$shannonPDF[1095] #1.738919
stats_of_whole_boot_instantaneous_50_2$b1[[2]][1095] #1.538075
stats_of_whole_boot_instantaneous_50_2$b2[[2]][1095] #1.839712

stats_of_whole_instantaneous_50_2$negentPDF[1] #0.8352903
stats_of_whole_boot_instantaneous_50_2$b2[[4]][1] #1.441464
stats_of_whole_boot_instantaneous_50_2$b1[[4]][1] #0.7284727
stats_of_whole_instantaneous_50_2$negentPDF[1095] #0.1072035
stats_of_whole_boot_instantaneous_50_2$b1[[4]][1095] #0.07247547
stats_of_whole_boot_instantaneous_50_2$b2[[4]][1095] #0.3258044

stats_of_whole_instantaneous_50_2$uniformKL_PDF[1] #0.966350_28
stats_of_whole_boot_instantaneous_50_2$b1[[5]][1] #0.5657428
stats_of_whole_boot_instantaneous_50_2$b2[[5]][1] #1.170918
stats_of_whole_instantaneous_50_2$uniformKL_PDF[1095] #0.2069911

```

```

stats_of_whole_boot_instantaneous_50_2$b2[[5]][1095] #0.3832459
stats_of_whole_boot_instantaneous_50_2$b1[[5]][1095] #0.08352609

min(stats_of_whole_instantaneous_50_2$uniformKL_PDF) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_instantaneous_50_2$uniformKL_PDF),stats_of_whole_instantaneous_50_2$uniformKL_PDF)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_instantaneous_50_2$uniformKL_PDF[min_index]
stats_of_whole_boot_instantaneous_50_2$b1[[5]][min_index]
stats_of_whole_boot_instantaneous_50_2$b2[[5]][min_index]

min(stats_of_whole_instantaneous_50_2$negentPDF) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_instantaneous_50_2$negentPDF),stats_of_whole_instantaneous_50_2$negentPDF)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_instantaneous_50_2$negentPDF[min_index]
stats_of_whole_boot_instantaneous_50_2$b1[[4]][min_index]
stats_of_whole_boot_instantaneous_50_2$b2[[4]][min_index]

min(stats_of_whole_instantaneous_50_2$uniformKL_Quantile) #0.1382115
min_index <-
  ↳ match(min(stats_of_whole_instantaneous_50_2$uniformKL_Quantile),stats_of_whole_instantaneous_50_2$uniformKL_Quantile)
filled$Date[min_index] #"1631-06-17"
stats_of_whole_instantaneous_50_2$uniformKL_Quantile[min_index]
stats_of_whole_boot_instantaneous_50_2$b1[[6]][min_index]
stats_of_whole_boot_instantaneous_50_2$b2[[6]][min_index]

max(stats_of_whole_instantaneous_50_2$shannonQuantile) #0.1382115
max_index <-
  ↳ match(max(stats_of_whole_instantaneous_50_2$shannonQuantile),stats_of_whole_instantaneous_50_2$shannonQuantile)
filled$Date[max_index] #"1631-06-17"
stats_of_whole_instantaneous_50_2$shannonQuantile[max_index]
stats_of_whole_boot_instantaneous_50_2$b1[[3]][max_index]
stats_of_whole_boot_instantaneous_50_2$b2[[3]][max_index]

max(stats_of_whole_instantaneous_50_2$shannonPDF) #0.1382115
max_index <-
  ↳ match(max(stats_of_whole_instantaneous_50_2$shannonPDF),stats_of_whole_instantaneous_50_2$shannonPDF)
filled$Date[max_index] #"1631-06-17"
stats_of_whole_instantaneous_50_2$shannonPDF[max_index]
stats_of_whole_boot_instantaneous_50_2$b1[[2]][max_index]
stats_of_whole_boot_instantaneous_50_2$b2[[2]][max_index]

ncol(filled_modified[,-1])
sum(population_specified_modified$Population*1000)
median(population_specified_modified$Population*1000)
mean(population_specified_modified$Population*1000)
sd(population_specified_modified$Population*1000)
sum(rowSums(filled_modified[,-1]))
mean(colSums(filled_modified[,-1])/(population_specified_modified$Population*1000))
sd(colSums(filled_modified[,-1])/(population_specified_modified$Population*1000))

```

```

#Effective number and log of non-zeroes
non_zeroes_cumulative <- rep(0,length(filled$Date))
eff_number_cumulative <- rep(0,length(filled$Date))
non_zeroes_instantaneous <- rep(0,length(filled$Date))
eff_number_instantaneous <- rep(0,length(filled$Date))
for(i in 1:length(filled$Date)){
  case_vec_cumulative <-
  ↪ as.numeric(colSums(filled_modified[1:i,-1]))/(population_specified_modified$Population*1000)
  case_vec_cumulative_norm <- case_vec_cumulative/sum(case_vec_cumulative)
  case_vec_instantaneous <-
  ↪ as.numeric(filled_modified[i,-1]))/(population_specified_modified$Population*1000)
  case_vec_instantaneous_norm <- case_vec_instantaneous/sum(case_vec_instantaneous)
  case_vec_cumulative_norm_nonzeroes <- case_vec_cumulative_norm[case_vec_cumulative_norm
  ↪ != 0]
  non_zeroes_cumulative[i] <- length(case_vec_cumulative_norm_nonzeroes)
  eff_number_cumulative[i] <- 1/sum(case_vec_cumulative_norm_nonzeroes^2)
  case_vec_instantaneous_norm_nonzeroes <-
  ↪ case_vec_instantaneous_norm[case_vec_instantaneous_norm != 0]
  non_zeroes_instantaneous[i] <- length(case_vec_instantaneous_norm_nonzeroes)
  eff_number_instantaneous[i] <- 1/sum(case_vec_instantaneous_norm_nonzeroes^2)
}
# plot.ts(log(non_zeroes_cumulative),ylim=c(2,4),col="red")
# lines(stats_of_whole_cumulative_50_2$shannonQuantile,col="black")
# plot.ts(log(non_zeroes_instantaneous),ylim=c(0,4),col="red")
# lines(stats_of_whole_instantaneous_50_2$shannonQuantile,col="black")
# plot.ts(log(eff_number_cumulative),ylim=c(2,4),col="red")
# lines(stats_of_whole_cumulative_50_2$shannonQuantile,col="black")
# plot.ts(log(eff_number_instantaneous),ylim=c(0,4),col="red")
# lines(stats_of_whole_instantaneous_50_2$shannonQuantile,col="black")

#1: sum, 2: shannonPDF, 3: shannonQuantile, 4: negentPDF, 5: uniformKL_PDF, 6:
  ↪ uniformKL_Quantile
#can plot against incidence ("sum") if you want

plot_df <-
  ↪ as.data.frame(cbind(stats_of_whole_boot_cumulative_50_2$b1[[6]],stats_of_whole_boot_cumulative_50_2
colnames(plot_df) <- c("b1","b2","observed")
plot1 <- ggplot(data=plot_df,aes(x=filled$Date,y=observed))+
  geom_line()+
  geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
  geom_line(aes(y=rep(0,length(filled$Date))),col="red")+
  coord_cartesian(ylim=c(0,0.35))+
  # coord_cartesian(ylim=c(0,0.5))+ #S. Marco
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous()+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
  ↪ element_blank(),panel.background =
  ↪ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
  labs(y="Negentropy (nats)",x="Date",title="Negentropy")
plot_df_2 <-
  ↪ as.data.frame(cbind(stats_of_whole_boot_cumulative_50_2$b1[[3]],stats_of_whole_boot_cumulative_50_2
colnames(plot_df_2) <- c("b1","b2","observed","eff")

```

```

plot2 <- ggplot(data=plot_df_2,aes(x=filled$Date,y=observed))+
  geom_line()+

  ↪ #geom_line(aes(y=rep(log(ncol(filled_modified[,-1])),length(filled$Date))),linetype="dashed")+
  geom_line(aes(y=rep(log(ncol(filled_modified[,-1])),length(filled$Date))),col="red")+
  geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
  #geom_line(aes(y=log(ef)),col="red")+
  # coord_cartesian(ylim=c(3.5,3.95))+ #S. Marco
  coord_cartesian(ylim=c(3.5,3.9))+
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous(name="Entropy (nats)", sec.axis=sec_axis(trans=~., name="Effective
  ↪ Number",labels = function(x) scales::comma(round(exp(x), accuracy = 1)))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
  ↪ element_blank(),panel.background =
  ↪ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
  labs(y="Entropy (nats)",x="Date",title="Entropy")
plot_ls <- list(plot2,plot1)
untitled_grid <- plot_grid(plotlist=plot_ls,labels=c("a","b"),label_size=12)
# now add the title
title <- ggdraw() +
  draw_label(
    "Quantile Function of Cumulative Mortality",
    fontface = 'bold',
    x = 0,
    hjust = 0
  ) +
  theme(
    # add margin on the left of the drawing canvas,
    # so title is aligned with left edge of first plot
    plot.margin = margin(0, 0, 0, 7)
  )
plot_grid(
  title, untitled_grid,
  ncol = 1,
  # rel_heights values control vertical title margins
  rel_heights = c(0.1, 1)
)
#ggsave("CumulativeQuantileGrid_Final.png",width=2100,height=1200,units="px")
#ggsave("CumulativeQuantileGrid_Final.jpg",width=2100,height=1200,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures/Figure2.png",width=2100,height=1200,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures/Figure2.jpg",width=2100,height=1200,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure2.png",width=2100,height=1200,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure2.jpg",width=2100,height=1200,units="px")
#
↪ ggsave("~/VenetianPlague_Entropy/SMarcoCumulativeQuantile.png",width=2100,height=1200,units="px")
#
↪ ggsave("~/VenetianPlague_Entropy/SMarcoCumulativeQuantile.jpg",width=2100,height=1200,units="px")

#1: sum, 2: shannonPDF, 3: shannonQuantile, 4: negentPDF, 5: uniformKL_PDF, 6:
↪ uniformKL_Quantile

```

```

plot_df <-
  ↪ as.data.frame(cbind(stats_of_whole_boot_cumulative_50_2$b1[[2]],stats_of_whole_boot_cumulative_50_2
colnames(plot_df) <- c("b1", "b2", "observed")
plot1 <- ggplot(data=plot_df,aes(x=filled$Date,y=observed))+
  geom_line()+

  ↪ #geom_line(aes(y=rep(log(ceiling(log2(ncol(filled_modified[,-1])))+1),length(filled$Date))),linet

  ↪ geom_line(aes(y=rep(log(ceiling(log2(ncol(filled_modified[,-1])))+1),length(filled$Date))),col="r
geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
coord_cartesian(ylim=c(1,2))+
# coord_cartesian(ylim=c(0,2))+ #S. Marco
scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
scale_y_continuous()+
theme_bw()+
theme(panel.grid.major = element_blank(), panel.grid.minor =
  ↪ element_blank(),panel.background =
  ↪ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
labs(y="Entropy (nats)",x="Date",title="Entropy")
plot_df_2 <-
  ↪ as.data.frame(cbind(stats_of_whole_boot_cumulative_50_2$b1[[4]],stats_of_whole_boot_cumulative_50_2
colnames(plot_df_2) <- c("b1", "b2", "observed")
plot2 <- ggplot(data=plot_df_2,aes(x=filled$Date,y=observed))+
  geom_line()+
  #geom_line(aes(y=funny_vec))+
  geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
  geom_line(aes(y=rep(0,length(filled$Date))),col="red")+
  coord_cartesian(ylim=c(0,0.45))+
  # coord_cartesian(ylim=c(0,1.5))+ #S. Marco
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous()+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
    ↪ element_blank(),panel.background =
    ↪ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
  #labs(y=expression(paste(D[KL], " (nats)")),x="Date",title="Negentropy")
  labs(y="Negentropy (nats)",x="Date",title="Negentropy")
# plot_df_3 <-
  ↪ as.data.frame(cbind(stats_of_whole_boot_cumulative_50_2$b1[[5]],stats_of_whole_boot_cumulative_50_2
# colnames(plot_df_3) <- c("b1", "b2", "observed")
# plot3 <- ggplot(data=plot_df_3,aes(x=filled$Date,y=observed))+
#   geom_line()+
#   geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
#   coord_cartesian(ylim=c(0,1))+
#   scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
#   scale_y_continuous()+
#   theme(plot.background=element_rect(fill="white"),axis.title=element_text(size=10))+
#   labs(y="KL Divergence",x="Date",title="KL Divergence from Uniform")
# plot_ls <- list(plot1,plot2,plot3)
# untitled_grid <- plot_grid(plotlist=plot_ls,labels=c("A", "B", "C"),label_size=12)
plot_ls <- list(plot1,plot2)
untitled_grid <- plot_grid(plotlist=plot_ls,labels=c("a", "b"),label_size=12)
# now add the title
title <- ggdraw() +

```

```

draw_label(
  "PDF of Cumulative Mortality",
  fontface = 'bold',
  x = 0,
  hjust = 0
) +
theme(
  # add margin on the left of the drawing canvas,
  # so title is aligned with left edge of first plot
  plot.margin = margin(0, 0, 0, 7)
)
plot_grid(
  title, untitled_grid,
  ncol=1,
  # rel_heights values control vertical title margins
  rel_heights = c(0.1, 1)
)
#ggsave("cumulative_50_2_2PDFGrid5.png",width=2100,height=150_20,units="px")
#ggsave("CumulativePDFGrid_Final.png",width=2100,height=1200,units="px")
#ggsave("CumulativePDFGrid_Final.jpg",width=2100,height=1200,units="px")
#
→ ggsave("~/VenetianPlague_Entropy/Figures/Figure3.png",width=2100,height=1200,units="px")
#
→ ggsave("~/VenetianPlague_Entropy/Figures/Figure3.jpg",width=2100,height=1200,units="px")
ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure3.png",width=2100,height=1200,units="px")
ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure3.jpg",width=2100,height=1200,units="px")
#
→ ggsave("~/VenetianPlague_Entropy/SMarcoCumulativePDF.png",width=2100,height=1200,units="px")
#
→ ggsave("~/VenetianPlague_Entropy/SMarcoCumulativePDF.jpg",width=2100,height=1200,units="px")

#1: sum, 2: shannonPDF, 3: shannonQuantile, 4: negentPDF, 5: uniformKL_PDF, 6:
→ uniformKL_Quantile
#can plot against incidence ("sum") if you want

plot_df <-
→ as.data.frame(cbind(stats_of_whole_boot_instantaneous_50_2$b1[[6]],stats_of_whole_boot_instantaneous_50_2$b2[[6]],stats_of_whole_boot_instantaneous_50_2$b3[[6]],stats_of_whole_boot_instantaneous_50_2$b4[[6]],stats_of_whole_boot_instantaneous_50_2$b5[[6]],stats_of_whole_boot_instantaneous_50_2$b6[[6]]))
colnames(plot_df) <- c("b1","b2","observed")
plot1 <- ggplot(data=plot_df,aes(x=filled$Date,y=observed))+
  geom_line(lwd=0.25)+
  geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
  geom_line(aes(y=rep(0,length(filled$Date)),col="red"))+
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous()+
  expand_limits(y=c(0,2))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
    → element_blank(),panel.background =
    → element_blank(),plot.background=element_rect(fill="white"),plot.title=element_text(size=12),axis.title=element_text(size=12))
  #labs(y=expression(D[K*L]),x="Date",title="Negentropy")
  labs(y="Negentropy (nats)",x="Date",title="Negentropy")
plot_df_2 <-
→ as.data.frame(cbind(stats_of_whole_boot_instantaneous_50_2$b1[[3]],stats_of_whole_boot_instantaneous_50_2$b2[[3]],stats_of_whole_boot_instantaneous_50_2$b3[[3]],stats_of_whole_boot_instantaneous_50_2$b4[[3]],stats_of_whole_boot_instantaneous_50_2$b5[[3]],stats_of_whole_boot_instantaneous_50_2$b6[[3]]))
colnames(plot_df_2) <- c("b1","b2","observed")
plot2 <- ggplot(data=plot_df_2,aes(x=filled$Date,y=observed))+
  geom_line(lwd=0.25)+
  geom_ribbon(aes(ymin=b1,ymax=b2),alpha=0.5)+
  geom_line(aes(y=rep(0,length(filled$Date)),col="red"))+
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous()+
  expand_limits(y=c(0,2))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
    → element_blank(),panel.background =
    → element_blank(),plot.background=element_rect(fill="white"),plot.title=element_text(size=12),axis.title=element_text(size=12))
  #labs(y=expression(D[K*L]),x="Date",title="Negentropy")
  labs(y="Negentropy (nats)",x="Date",title="Negentropy")

```

```

colnames(plot_df_2) <- c("b1", "b2", "observed", "nonzeroes")
plot2 <- ggplot(data=plot_df_2, aes(x=filled$Date, y=observed))+
  #geom_line(aes(y=nonzeroes), lwd=0.25, col="red", alpha=0.75)+ #comment out if don't like
  geom_line(lwd=0.25)+
  geom_line(aes(y=rep(log(ncol(filled_modified[, -1])), length(filled$Date))), col="red")+
  geom_ribbon(aes(ymin=b1, ymax=b2), alpha=0.5)+
  scale_x_date(date_breaks="12 months", date_labels="%m-%Y")+
  scale_y_continuous()+
  expand_limits(y=c(2, 4))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
    ↪ element_blank(), panel.background =
    ↪ element_blank(), plot.background=element_rect(fill="white"), plot.title=element_text(size=12), axis.
  labs(y="Entropy (nats)", x="Date", title="Entropy")
plot_ls <- list(plot2, plot1)
untitled_grid <- plot_grid(plotlist=plot_ls, labels=c("a", "b"), label_size=12)
# now add the title
title <- ggdraw() +
  draw_label(
    "Quantile Function of Instantaneous Mortality",
    fontface = 'bold',
    x = 0,
    hjust = 0
  ) +
  theme(
    # add margin on the left of the drawing canvas,
    # so title is aligned with left edge of first plot
    plot.margin = margin(0, 0, 0, 7)
  )
plot_grid(
  title, untitled_grid,
  ncol = 1,
  # rel_heights values control vertical title margins
  rel_heights = c(0.1, 1)
)
#ggsave("InstantaneousQuantileGrid_Final_2.png", width=2100, height=1200, units="px")
#ggsave("InstantaneousQuantileGrid_Final_2.jpg", width=2100, height=1200, units="px")
#ggsave("InstantaneousQuantileGrid_Final.jpg", width=2100, height=1200, units="px")
#ggsave("InstantaneousQuantileGrid_Final_3.jpg", width=2100, height=1200, units="px")
ggsave("InstantaneousQuantileGrid_Final_4.jpg", width=2100, height=1200, units="px")

#1: sum, 2: shannonPDF, 3: shannonQuantile, 4: negentPDF, 5: uniformKL_PDF, 6:
↪ uniformKL_Quantile
plot_df <-
↪ as.data.frame(cbind(stats_of_whole_boot_instantaneous_50_2$b1[[2]], stats_of_whole_boot_instantaneous
colnames(plot_df) <- c("b1", "b2", "observed")
plot1 <- ggplot(data=plot_df, aes(x=filled$Date, y=observed))+
  geom_ribbon(aes(x=filled$Date, ymin=b1, ymax=b2), alpha=0.5)+
  geom_line(lwd=0.25)+
  ↪ geom_line(aes(y=rep(log(ceiling(log2(ncol(filled_modified[, -1])))+1), length(filled$Date))), col="r
  scale_x_date(date_breaks="12 months", date_labels="%m-%Y")+
  scale_y_continuous()+
  expand_limits(y=c(0, 2))+

```

```

theme_bw()+
theme(panel.grid.major = element_blank(), panel.grid.minor =
  ↳ element_blank(),panel.background =
  ↳ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
labs(y="Entropy (nats)",x="Date",title="Entropy")
plot_df_2 <-
  ↳ as.data.frame(cbind(stats_of_whole_boot_instantaneous_50_2$b1[[4]],stats_of_whole_boot_instantaneous_50_2$b2[[4]],stats_of_whole_boot_instantaneous_50_2$observed))
colnames(plot_df_2) <- c("b1", "b2", "observed")
plot2 <- ggplot(data=plot_df_2,aes(x=filled$Date,y=observed))+
  geom_ribbon(aes(x=filled$Date,ymin=b1,ymax=b2),alpha=0.5)+
  geom_line(lwd=0.25)+
  geom_line(aes(y=rep(0,length(filled$Date))),col="red")+
  scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
  scale_y_continuous()+
  coord_cartesian(ylim=c(0,1.5))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor =
    ↳ element_blank(),panel.background =
    ↳ element_blank(),plot.background=element_rect(fill="white"),axis.title=element_text(size=10),plot.
labs(y="Negentropy (nats)",x="Date",title="Negentropy")
# plot_df_3 <-
  ↳ as.data.frame(cbind(stats_of_whole_boot_instantaneous_50_2$b1[[5]],stats_of_whole_boot_instantaneous_50_2$b2[[5]],stats_of_whole_boot_instantaneous_50_2$observed))
# colnames(plot_df_3) <- c("b1", "b2", "observed")
# plot3 <- ggplot(data=plot_df_3,aes(x=filled$Date,y=observed))+
#   geom_ribbon(aes(x=filled$Date,ymin=b1,ymax=b2),alpha=0.5)+
#   geom_line(lwd=0.25)+
#   scale_x_date(date_breaks="12 months",date_labels="%m-%Y")+
#   scale_y_continuous()+
#   expand_limits(y=c(0,1.8))+
#   theme(plot.background=element_rect(fill="white"),axis.title=element_text(size=10))+
#   labs(y="KL Divergence",x="Date",title="KL Divergence from Uniform")
plot_ls <- list(plot1,plot2)
untitled_grid <- plot_grid(plotlist=plot_ls,labels=c("a","b"),label_size=12)
# now add the title
title <- ggdraw() +
  draw_label(
    "PDF of Instantaneous Mortality",
    fontface = 'bold',
    x = 0,
    hjust = 0
  ) +
  theme(
    # add margin on the left of the drawing canvas,
    # so title is aligned with left edge of first plot
    plot.margin = margin(0, 0, 0, 7)
  )
plot_grid(
  title, untitled_grid,
  ncol=1,
  # rel_heights values control vertical title margins
  rel_heights = c(0.1, 1)
)
#ggsave("InstantaneousPDFGrid_Final_2.png",width=2100,height=1200,units="px")
#ggsave("InstantaneousPDFGrid_Final_2.jpg",width=2100,height=1200,units="px")

```

```

#ggsave("InstantaneousPDFGrid_Final.jpg",width=2100,height=1200,units="px")
#ggsave("InstantaneousPDFGrid_Final_3.jpg",width=2100,height=1200,units="px")
ggsave("InstantaneousPDFGrid_Final_4.jpg",width=2100,height=1200,units="px")

stat_of_interest <- 3 #1: sum, 2: shannonPDF, 3: shannonQuantile, 4: negentPDF, 5:
  ↪ uniformKL_PDF, 6: uniformKL_Quantile
b1_plot_mat <- matrix(NA,nrow=1095,ncol=6) #6 = number of stats, not sestieri
b2_plot_mat <- matrix(NA,nrow=1095,ncol=6)
observed_plot_mat <- matrix(NA,nrow=1095,ncol=6)

for(i in 1:6){ #6 = number of sestieri
  b1_plot_mat[,i] <- stats_by_sestiere_boot_cumulative_50_2[[i]]$b1[,stat_of_interest]
  b2_plot_mat[,i] <- stats_by_sestiere_boot_cumulative_50_2[[i]]$b2[,stat_of_interest]
  observed_plot_mat[,i] <- stats_by_sestiere_cumulative_50_2[[i]][[stat_of_interest]]
}

time_series <- 1:1095
series_names <- unique(sestiere_df_modified$Sestiere)

# Ensure matrices are data.frames
lower_df <- as.data.frame(b1_plot_mat)
upper_df <- as.data.frame(b2_plot_mat)
observed_df <- as.data.frame(observed_plot_mat)

# Add time column
lower_df$time <- upper_df$time <- observed_df$time <- time_series

# Pivot longer
lower_long <- pivot_longer(lower_df, -time, names_to = "series", values_to = "lower")
upper_long <- pivot_longer(upper_df, -time, names_to = "series", values_to = "upper")
obs_long <- pivot_longer(observed_df, -time, names_to = "series", values_to =
  ↪ "observed")

# Combine all into one long dataframe
plot_df <- lower_long %>%
  left_join(upper_long, by = c("time", "series")) %>%
  left_join(obs_long, by = c("time", "series")) %>%
  mutate(series = factor(series, levels = paste0("V", 1:6), labels = series_names))

maxima_vec <- rep(0,length(unique(sestiere_df_modified$Sestiere)))
for(s in unique(sestiere_df_modified$Sestiere)){
  maxima_vec[match(s,unique(sestiere_df_modified$Sestiere))] <-
  ↪ log(length(sestiere_df_modified$Parish[sestiere_df_modified$Sestiere==s]))
}

maxima_column <- rep(maxima_vec,nrow(filled_modified))
plot_df$maxima <- maxima_column

ggplot(plot_df, aes(x = rep(filled$Date,each=6), group = series, color = series, fill =
  ↪ series)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.4, color = NA) +
  geom_line(aes(y = lower), linetype = "dashed", linewidth = 0.25) +
  geom_line(aes(y = upper), linetype = "dashed", linewidth = 0.25) +
  geom_line(aes(y = observed), linewidth = 0.5) +

```

```

geom_line(aes(y = maxima),linetype="dashed",linewidth=0.75) +
scale_color_manual(values = c("green", "red", "blue", "yellow", "orange", "pink")) +
scale_fill_manual(values = c("green", "red", "blue", "yellow", "orange", "pink")) +
coord_cartesian(ylim=c(0.5,2.5))+
scale_y_continuous(name="Entropy (nats)", sec.axis=sec_axis(trans=~., name="Effective
↪ Number",labels = function(x) scales::comma(exp(x), accuracy = 0.1)))+
theme_bw()+
theme(panel.grid.major = element_blank(), panel.grid.minor =
↪ element_blank(),panel.background =
↪ element_blank(),plot.background=element_rect(fill="white")) +
labs(
  title = "Quantile Function of Cumulative Mortality by Sestiere",
  #title = names(stats_of_whole_cumulative_50_2)[stat_of_interest],
  x = "Date",
  y = "Entropy (nats)",
  color = "Sestiere",
  fill = "Sestiere"
)
#ggsave("CumulativeQuantile_KL_Sestiere_Final.png",width=2100,height=1500,units="px")
↪ #needs to be .jpg
#ggsave("CumulativeQuantile_KL_Sestiere_Final.jpg",width=2100,height=1500,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures/Figure4.png",width=2100,height=1500,units="px")
#ggsave("~/VenetianPlague_Entropy/Figures/Figure4.jpg",width=2100,height=1500,units="px")
ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure4.png",width=2100,height=1500,units="px")
ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure4.jpg",width=2100,height=1500,units="px")

```

PDF and Quantile Graphs

In this section, we will generate the PDF and Quantile Graphs shown in Figure 1.

First, we define our functions.

```

library(ggplotify)
library(ggplot2)
library(tmap)
library(sf)

# day <- sample(365:1095,1)
# case_vec <-
↪ as.numeric(colSums(filled_modified[1:day,-1]))/(population_specified_modified$Population*1000)
# case_vec_norm <- case_vec/sum(case_vec)
# q_func <- sort(case_vec_norm)

sf_use_s2(FALSE)

end_of_month_indices <- c(which(day(filled$Date) == 1)[-1] - 1, 1095)

negent_new <- function (x, K = ceiling(log2(length(x))) + 1){
  p <- table(cut(x, breaks = K))/length(x)
  sigma2 <- var(x)
  #p <- p[(p > 0)]
  bins <- seq(min(x), max(x), length.out = K + 1)
  bins2 <- rep(0, K)
  for(i in 1:K){
    bins2[i] <- (bins[i] + bins[i+1]) / 2
  }
}

```

```

}
y <- dnorm(bins2, mean = mean(x), sd = sd(x))
y <- y / sum(y)
if(length(which(is.na(y))) == K){
  y <- rep(0, K)
}
if(length(as.numeric(p)) != length(y)){
  print("Something went wrong")
  print(paste("p", length(as.numeric(p))))
  print(as.numeric(p))
  print(paste("y", length(y)))
  print(y)
} else if(anyNA(as.numeric(p))){
  print(paste("p had an NA", as.numeric(p)))
} else if(anyNA(y)){
  print(paste("y had an NA", y))
}
return(KL_new(as.numeric(p), y))
#return(log(2 * pi * exp(1)*sigma2)*1/2 + sum(p * log(p)))
}

PDF_approx_new <- function (x, K = ceiling(log2(length(x))) + 1, epsilon = 1e-05){
  p <- table(cut(x+epsilon, breaks = K))/length(x)
  #p <- p[(p > 0)]
  return(p)
  #return(log(2 * pi * exp(1)*sigma2)*1/2 + sum(p * log(p)))
}

```

While we use histogram approximation of the PDF with bin number determined by Sturges' rule, for the purposes of illustration we use an Epanechnikov kernel with 512 bins in this figure.

```

PDF_approx_epanechnikov <- function (cv){
  #kern <- density(cv,n=512,from=0,to=max(cv),kernel="epanechnikov",bw="bcv")$y
  kern <- density(cv, n = 512, from = min(cv), to = max(cv), bw = "nrd0")$y
  kern <- kern / sum(kern)
  return(kern)
}

normal_approx <- function (x, K = ceiling(log2(length(x))) + 1, epsilon = 1e-05){
  bins <- seq(min(x), max(x), length.out = K + 1)
  bins2 <- rep(0, K)
  for(i in 1:K){
    bins2[i] <- (bins[i] + bins[i + 1]) / 2
  }
  y <- dnorm(bins2, mean = mean(x), sd = sd(x))
  y <- y / sum(y)
  return(y)
  #return(log(2 * pi * exp(1)*sigma2)*1/2 + sum(p * log(p)))
}

#months_vec <- c(12,24,34)
days_vec <- c(7, end_of_month_indices[22], end_of_month_indices[36])

#par(mfrow=c(1,2)) #create matrix
new_negent <- rep(0, length(days_vec))

```

```

old_negent <- rep(0, length(days_vec))
shannon_normal <- rep(0, length(days_vec))

quantile_graph_list <- vector(mode= "list", length = 3)
PDF_graph_list <- vector(mode="list", length = 3)
grobbed_tmap_list <- vector(mode = "list", length = 3)
title_list <- vector(mode = "list", length = 3)

load(file = "~/Downloads/plot_list7.RData")

# for(i in 1:length(end_of_month_indices)){
#   day <- end_of_month_indices[i]
#   case_vec <-
#     ↪ as.numeric(colSums(filled_modified[1:day,-1]))/(population_specified_modified$Population*1000)
#   var_mean <- sd(case_vec)/mean(case_vec)
#   ↪ plot(PDF_approx_epanechnikov(case_vec),main=paste(filled$Date[day],round(stats_of_whole_cumulative_
# }

coeff_var_mat <- matrix(NA, nrow = 999, ncol = 3)

for(i in 1:999){
  indices <- sample(1:ncol(filled_modified[, -1]), replace = TRUE)
  bootied_data <- filled_modified[, -1][, indices]
  pop_spec <- population_specified_modified[indices,]
  for(j in days_vec){
    day <- j
    case_vec <- as.numeric(colSums(bootied_data[1:day,])) / (pop_spec$Population * 1000)
    var_mean <- sd(case_vec) / mean(case_vec)
    coeff_var_mat[i, match(j, days_vec)] <- var_mean
  }
}

sort(coeff_var_mat[,1], decreasing=FALSE)[25] #June 30, 1630 low
sort(coeff_var_mat[,1], decreasing=FALSE)[975] #June 30, 1630 high
sort(coeff_var_mat[,2], decreasing=FALSE)[25] #December 31, 1630 low
sort(coeff_var_mat[,2], decreasing=FALSE)[975] #December 31, 1630 high
sort(coeff_var_mat[,3], decreasing=FALSE)[25] #October 31, low
sort(coeff_var_mat[,3], decreasing=FALSE)[975] #October 31, 1630 high

for(i in days_vec){
  day <- i
  case_vec <- as.numeric(colSums(filled_modified[1:day,-1])) /
  ↪ (population_specified_modified$Population * 1000)
  var_mean <- sd(case_vec) / mean(case_vec)
  print(var_mean)
  case_vec_norm <- case_vec / sum(case_vec)
  q_func <- sort(case_vec_norm)
  pdf_vec <- as.numeric(PDF_approx_new(case_vec))
  normal <- normal_approx(case_vec)
  shannon_normal[match(i, days_vec)] <- -1 * sum(normal[normal!=0] *
  ↪ log(normal[normal!=0]))
  # plot(pdf_vec,main=paste(filled$Date[day],round(var_mean,3))) # PDF
  # lines(normal_approx(case_vec),col="red")
  # barplot(q_func) #quantile

```

```

# plot(PDF_approx_epanechnikov(case_vec))
new_negent[match(i,days_vec)] <- negent_new(case_vec)
old_negent[match(i,days_vec)] <- negent(case_vec)
max_ent_uniform <- rep(1/length(case_vec_norm),length(case_vec_norm))
q_func_df <- as.data.frame(cbind(q_func,max_ent_uniform))
if(match(i,days_vec)==1){
  quantile_graph <- ggplot(data = q_func_df, aes(y = q_func, x = seq(1,
→ ncol(filled_modified[, -1])))) +
  geom_bar(stat = "identity") +
  geom_bar(aes(y = max_ent_uniform), stat = "identity", fill = "red", alpha = 0.25) +
  labs(y = "Normalized Mortality", x = "Parishes Ranked by Mortality") +
  coord_cartesian(ylim = c(0, 0.15)) +
  theme_bw() +
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
→ panel.background = element_blank(), axis.text.x = element_blank(), axis.ticks.x
→ = element_blank(), plot.margin = margin(5.5, 5.5, 5.5, 5.5), axis.title =
→ element_text(size = 10))
  quantile_graph <- quantile_graph + ggtitle("Quantile Function")
} else {
  quantile_graph <- ggplot(data = q_func_df, aes(y = q_func, x =
→ seq(1,ncol(filled_modified[, -1])))) +
  geom_bar(stat = "identity") +
  geom_bar(aes(y = max_ent_uniform), stat = "identity", fill = "red", alpha = 0.25) +
  labs(y = "Normalized Mortality", x = "Parishes Ranked by Mortality") +
  coord_cartesian(ylim = c(0, 0.04)) +
  theme_bw() +
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
→ panel.background = element_blank(), axis.text.x = element_blank(), axis.ticks.x
→ = element_blank(), plot.margin = margin(5.5, 5.5, 5.5, 5.5), axis.title =
→ element_text(size = 10))
}
K <- ceiling(log2(length(case_vec))) + 1
bins <- seq(min(case_vec), max(case_vec), length.out = K + 1)
bins2 <- rep(0, K)
for(j in 1:K){
  bins2[j] <- (bins[j] + bins[j+1]) / 2
}
bins3 <- seq(from = min(case_vec), to = max(case_vec), length.out = 512)
pdf_vec_2 <- PDF_approx_epanechnikov(case_vec)
max_ent_normal <- dnorm(bins3, mean = mean(case_vec), sd = sd(case_vec))
max_ent_normal <- max_ent_normal / sum(max_ent_normal)
pdf_df <- data.frame(x = bins3, y = pdf_vec_2, y2 = max_ent_normal)
if(match(i,days_vec) == 1){
  PDF_graph <- ggplot(data = pdf_df, aes(x = x, y = y)) +
  geom_density(stat = "identity", fill = "darkgrey", alpha = 0.5, lwd = 0.25) +
  geom_density(aes(y = y2), stat = "identity", fill = "red", alpha = 0.25, lwd =
→ 0.25) +
  labs(y = "Normalized Parish Count", x = "Cumulative Mortality") +
  coord_cartesian(ylim = c(0, 0.01)) +
  theme_bw() +
  theme(panel.grid.major = element_blank(), panel.grid.minor =
→ element_blank(), panel.background = element_blank(), plot.margin = margin(5.5,
→ 5.5, 5.5, 5.5), axis.title = element_text(size = 10))
  PDF_graph <- PDF_graph + ggtitle("PDF")
}

```

```

} else {
PDF_graph <- ggplot(data=pdf_df,aes(x = x, y = y))+
  geom_density(stat = "identity",fill = "darkgrey",alpha = 0.5,lwd = 0.25)+
  geom_density(aes(y = y2),stat = "identity",fill = "red", alpha = 0.25, lwd = 0.25)+
  labs(y = "Normalized Parish Count",x = "Cumulative Mortality")+
  coord_cartesian(ylim = c(0, 0.006))+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
  ↪ panel.background = element_blank(), plot.margin = margin(5.5, 5.5, 5.5, 5.5),
  ↪ axis.title = element_text(size = 10))
}
quantile_graph_list[[match(i, days_vec)]] <- quantile_graph

if(match(i,days_vec) == 1){
  tmap_plot <- plot_list7[[match(i, days_vec)]] +
    tm_layout(
      main.title="    Choropleth Map",
      main.title.size=1.1,
      #legend.text.size = .55,
      outer.margins = c(0, 0, 0, 0),
      inner.margins = c(0, 0, 0, 0),
      frame = FALSE
    )
} else{
  tmap_plot <- plot_list7[[match(i, days_vec)]] +
    tm_layout(
      main.title = " ",
      main.title.size = 1.1,
      #legend.text.size = .55,
      outer.margins = c(0, 0, 0, 0),
      inner.margins = c(0, 0, 0, 0),
      frame = FALSE
    )
}
grobbed_tmap_plot <- tmap_grob(tmap_plot)

grobbed_tmap_list[[match(i, days_vec)]] <- grobbed_tmap_plot

PDF_graph_list[[match(i, days_vec)]] <- PDF_graph

PDF_and_quantile_graph <- plot_grid(plotlist = list(PDF_graph,quantile_graph))

#untitled_grid <- plot_grid(plotlist=list(PDF_and_quantile_graph,heatmap_plot))

title <- ggdraw() +
  draw_label(
    filled$Date[day],
    fontface = 'bold',
    x = 0,
    hjust = 0
  ) +
  theme(
    # add margin on the left of the drawing canvas,
    # so title is aligned with left edge of first plot

```

```

    plot.margin = margin(0, 0, 0, 7)
  )
  title_list[[match(i,days_vec)]] <- title
  overall_graph <- plot_grid(
    title, PDF_and_quantile_graph,
    ncol = 1, nrow = 2,
    # rel_heights values control vertical title margins
    rel_heights = c(0.1, 1)
  )
  print(overall_graph)
}
par(mfrow = c(1, 1)) #reset

master_plot_list <- vector(mode = "list", length = 9)
master_plot_list_1 <- vector(mode = "list", length = 3)
master_plot_list_2 <- vector(mode = "list", length = 3)
master_plot_list_3 <- vector(mode = "list", length = 3)

for(k in 1:length(days_vec)){
  master_plot_list[[k]] <- grobbed_tmap_list[[k]]
  master_plot_list_1[[k]] <- grobbed_tmap_list[[k]]
}

for(k in 4:6){
  master_plot_list[[k]] <- quantile_graph_list[[k-3]]
  master_plot_list_2[[k-3]] <- quantile_graph_list[[k-3]]
}
for(k in 7:9){
  master_plot_list[[k]] <- PDF_graph_list[[k-6]]
  master_plot_list_3[[k-6]] <- PDF_graph_list[[k-6]]
}

plot_grid_1 <- plot_grid(plotlist = master_plot_list_1, ncol = 3, nrow = 1, align = "hv",
  ↪ axis = "tblr", labels = c("a", "", ""), label_size = 12)
plot_grid_2 <-
  ↪ plot_grid(plotlist=master_plot_list_2,ncol=3,nrow=1,align="hv",axis="tblr",labels=c("b","", ""),label
plot_grid_3 <-
  ↪ plot_grid(plotlist=master_plot_list_3,ncol=3,nrow=1,align="hv",axis="tblr",labels=c("c","", ""),label

plot_grid(plot_grid_1,plot_grid_2,plot_grid_3,nrow=3,align="hv",axis="tblr")

plot_grid(plotlist = master_plot_list, ncol = 3, nrow = 3, align = 'hv', axis = 'tblr',
  ↪ labels = c("a", "", "", "b", "", "", "c", "", ""), label_size = 12)
#ggsave("Gridded_Incidence_Map_Final.png",width=2900,height=1800,units="px")
#ggsave("Gridded_Incidence_Map_Final_10.png", width = 3000, height = 1800, units = "px")
#ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure1v2.png", width = 3000, height =
  ↪ 1800, units = "px")
#ggsave("~/VenetianPlague_Entropy/Figures_Final/Figure1v2.jpg", width = 3000, height =
  ↪ 1800, units = "px")

# plot.ts(new_negent,col="red",ylim=c(0,2))
# lines(old_negent)

```

Mortality Graphs

In this section, we generate the mortality graphs seen in Supplementary Figure 7.

```
library(cowplot)

outliers <- c("S.Polo", "S.Stae", "S.Marco") #No cases between 9/1/30 and 12/31/30,
  ↪ outbreak time

filled_nodate <- filled[,-1]
outlier_indices <- which(colnames(filled_nodate) %in% outliers)
total_deaths <- colSums(filled_nodate[1:1095,])

print(colnames(filled_nodate) == population_specified$Parish)

export_df <- as.data.frame(cbind(1:ncol(filled_nodate), total_deaths,
  ↪ population_specified$Population * 1000))
write.csv(export_df, "total_deaths.csv")

print(outlier_indices)

mort_df <- as.data.frame(matrix(NA, ncol = 3, nrow = 1095))
colnames(mort_df) <- c("Date", "Mort", "CumMort")
mort_df$Date <- as.Date(filled$Date)

for(i in 1:1095){
  mort_df[i,2] <- sum(as.numeric(filled_nodate[i,]) / (population_specified$Population *
  ↪ 1000)) / ncol(filled_nodate)
  mort_df[i,3] <- sum(as.numeric(colSums(filled_nodate[1:i,]))) /
  ↪ (population_specified$Population * 1000)) / ncol(filled_nodate)
}

regions <- tibble(x1 = as.Date("1630-09-01"), x2 = as.Date("1630-12-31"), y1 = -Inf, y2 =
  ↪ +Inf)

daily_mort_plot <- ggplot(mort_df)+
  geom_rect(data = regions, inherit.aes = FALSE, mapping = aes(xmin = x1, xmax = x2, ymin
  ↪ = y1, ymax = y2), color = "transparent", fill = "blue", alpha = 0.2)+
  geom_line(aes(x = as.Date(Date), y = Mort))+
  labs(x = "Date", y = "Mean Daily Parish Mortality", title = "Daily Mortality")+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
  ↪ axis.title = element_text(size = 10))

cumulative_mort_plot <- ggplot(mort_df)+
  geom_rect(data = regions, inherit.aes = FALSE, mapping = aes(xmin = x1, xmax = x2, ymin
  ↪ = y1, ymax = y2), color = "transparent", fill = "blue", alpha = 0.2)+
  geom_line(aes(x = as.Date(Date), y = CumMort))+
  labs(x = "Date", y = "Mean Cumulative Parish Mortality", title = "Cumulative
  ↪ Mortality")+
  theme_bw()+
  theme(panel.grid.major = element_blank(), panel.grid.minor = element_blank(),
  ↪ axis.title = element_text(size = 10))

untitled_grid <- plot_grid(plotlist = list(daily_mort_plot, cumulative_mort_plot), labels
  ↪ = c("a", "b"), label_size = 12)
```

```

title <- ggdraw() +
  draw_label(
    "Mortality Over Time",
    fontface = 'bold',
    x = 0,
    hjust = 0
  ) +
  theme(
    # add margin on the left of the drawing canvas,
    # so title is aligned with left edge of first plot
    plot.margin = margin(0, 0, 0, 7)
  )
plot_grid(
  title, untitled_grid,
  ncol = 1,
  # rel_heights values control vertical title margins
  rel_heights = c(0.1, 1)
)
#ggsave("SuppFig7.jpg", width = 2200, height = 1080, units = "px")

sestiere_vec <- c()
for(i in unique(sestiere_df_modified$Sestiere)){
  sestiere_vec <- c(sestiere_vec,
    ↪ length(sestiere_df_modified$Parish[sestiere_df_modified$Sestiere==i]))
}
print(sum(sestiere_vec) / length(sestiere_vec))
print(sd(sestiere_vec))

final_mortality <- as.numeric(colSums(filled_modified[1:1095,-1]) /
  ↪ (population_specified_modified$Population * 1000))
mean_mortality <- c()
for(i in unique(sestiere_df_modified$Sestiere)){
  sestiere_mortality <- final_mortality[which(population_specified_modified$Parish %in%
  ↪ sestiere_df_modified$Parish[sestiere_df_modified$Sestiere == i])]
  mean_mortality <- c(mean_mortality, sum(sestiere_mortality) /
  ↪ length(sestiere_mortality))
  print(sd(sestiere_mortality))
}
print(sum(mean_mortality) / length(mean_mortality))
print(sd(mean_mortality))

```